THE ODIN PROJECT

JAVASCRIPT

Full Stack JavaScript Learning Path

Presented by Tejus Sanly

Subject: Computer Systems | Group: 4230z

Course roadmap

From structured code to complete interactive applications

01 Foundations of architecture

Objects, constructors, factory functions, classes and modules.

02 Modern tooling

npm, Webpack, JSON, linting and form validation.

03 Advanced JavaScript

Promises, async/await, APIs, testing and recursion.

04 Computer science & projects

Algorithms, data structures, Git and full applications.

Organizing JavaScript code

Reusable code is easier to understand, test and maintain

Objects and constructors

Group related data and behaviors inside reusable structu

Factories, classes & modules

Create instances and split functionality into maintainable

```
class Book {
  constructor(title) {
    this.title = title;
  }
}
const book = new Book("Dune");
```

Tools for modern development

Dependencies, bundling and code quality

npm

Install and manage third-party JavaScript packages.

Webpack & ES6 modules

Bundle multiple code files for the browser.

JSON

Exchange structured data between applications and APIs.

Linting & forms

Catch problems early and validate user inputs.

Asynchronous JavaScript & APIs

Work with operations that finish later, without freezing the page

Promises

Represent a future result or failure.

Fetch API

Request real-world data from a server.

```
async function loadWeather() {
  const response = await fetch(url);
  const data = await response.json();
  return data;
}
```

Build practical browser projects

Projects reinforce architecture, DOM work and user interaction

Library

Store, display and update a personal book collection.

Tic Tac Toe

Manage game state, rules and player turns.

Restaurant Page

Generate page content using modules and Webpack.

Todo List

Organize tasks, projects, dates and persistence.

Testing & debugging

Check expected behavior before releasing code

Testing basics

Write automated checks for functions and edge cases.

Testing practice

Use assertions to compare expected and actual output.

More testing

Build confidence through repeatable tests.

Form validation

Give users clear feedback on invalid form entries.

Computer science essentials

Understand the efficiency and structure behind applications

Recursion

Solve a problem by reducing it to smaller versions.

Big O complexity

Compare time and memory requirements as inputs grow.

Core data structures

Linked lists, hash maps and binary search trees.

Algorithms

Traversal, searching and problem-solving strategies.

Applied projects & collaboration

Turn course knowledge into portfolio-ready work

Weather App

Fetch and display live data from a weather API.

Battleship

Combine game logic, modular code and automated tests.

Knights Travails

Find paths through a graph using search algorithms.

Intermediate Git

Collaborate using branches, remotes and workflows.

Key takeaways

JavaScript turns static pages into complete, interactive applications

- ✓ Write reusable code with objects, modules and classes
- ✓ Build apps with modern tools and external APIs
- ✓ Test programs and reason about algorithms
- ✓ Showcase finished projects on GitHub

Source: theodinproject.com/paths/full-stack-javascript/courses/javascript